adc

FLUTTER MEETUP - 6 JANUARY 2026

OPENING WORD

adc

SPONSOR

الهيئات الطلابية

Student Associations

adc

Flutter MEETUP

Mahmood Tantora

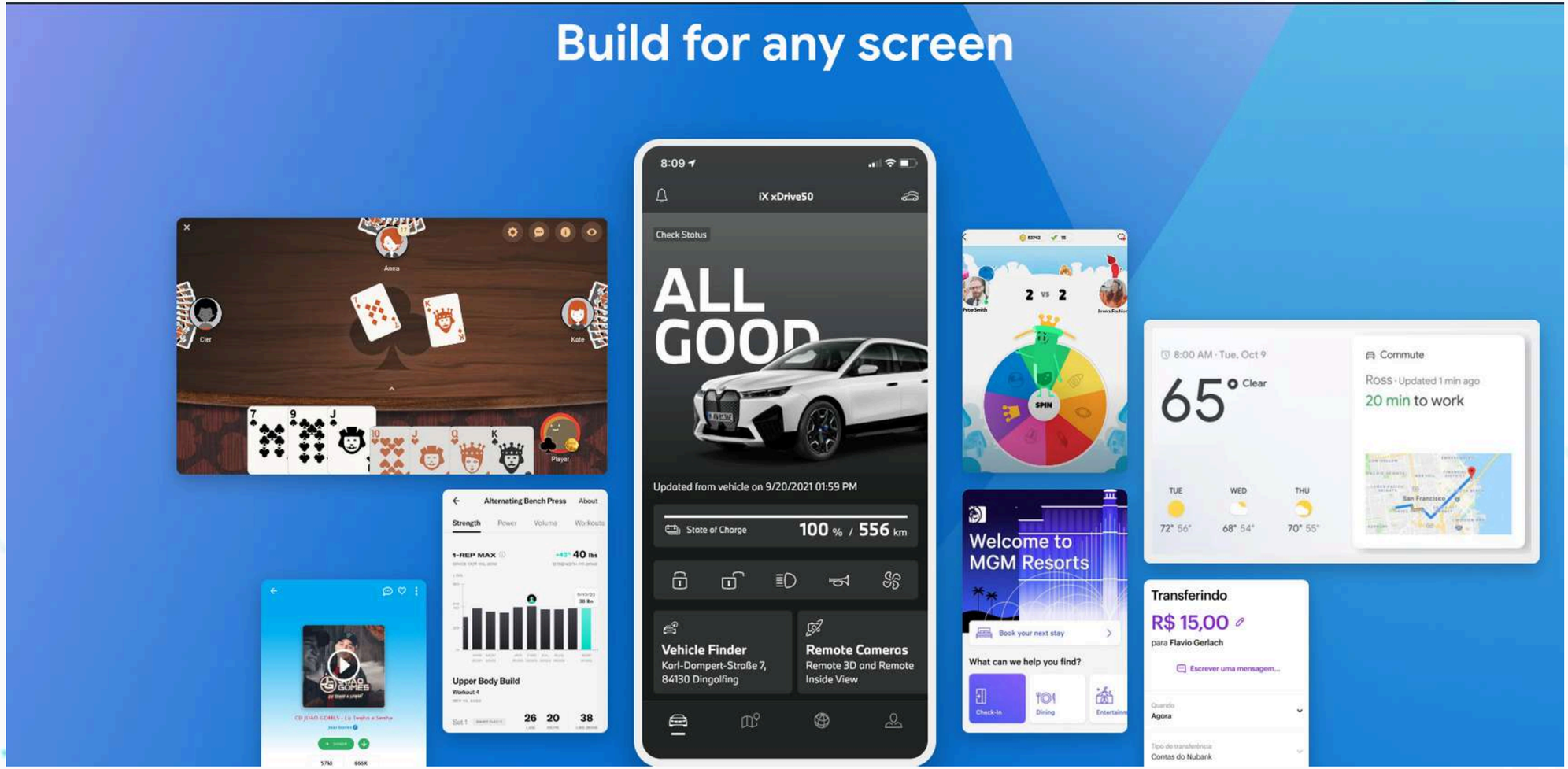
Senior Mobile Developer

Flutter Beyond Mobile: Flutter Web AI-Powered Development

FLUTTER FLASH

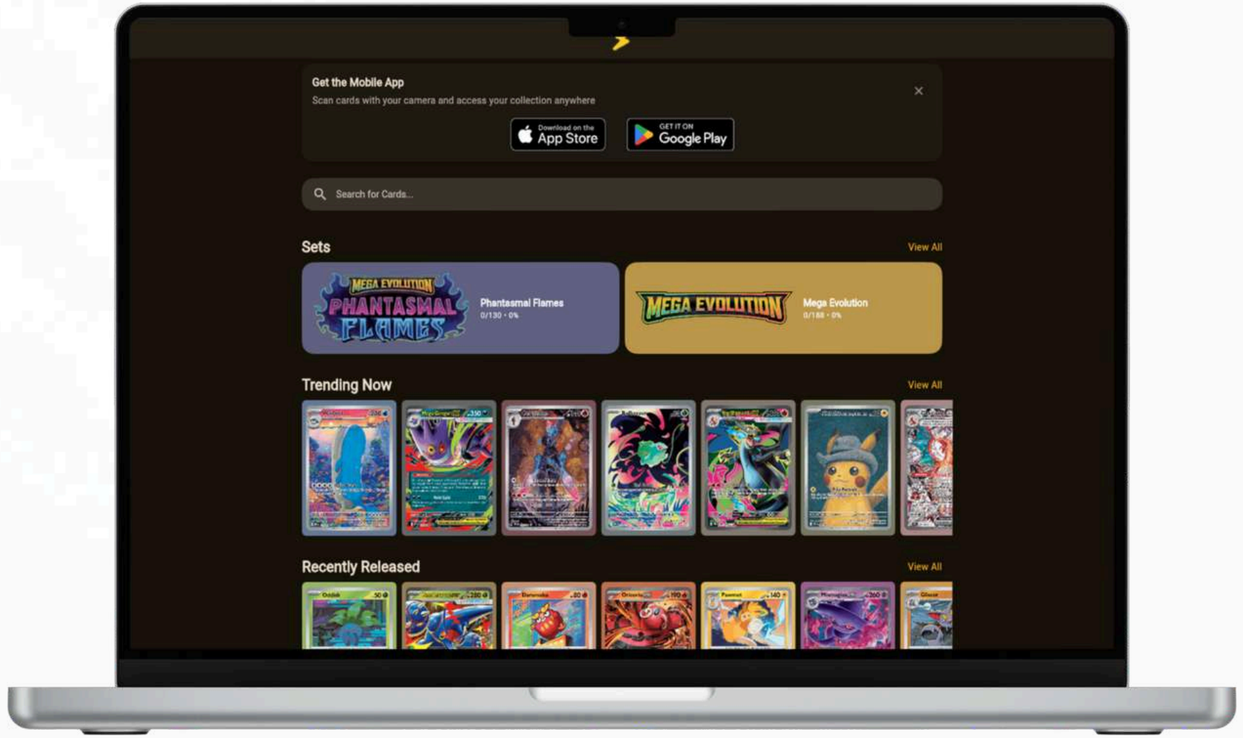
- Flutter Platform Channels
- Flutter Testing
- Rendering
- Code Generators
- Flutter Hardware Connection

Flutter Isn’t only mobile



Flutter Web

Flutter Web allows developers to reuse Flutter skills to build web dashboards, admin panels, and MVPs efficiently.



adc

Why Flutter Developers Need Web?

Faster development for business tools

Shared logic with mobile apps

One ecosystem instead of multiple stacks

Responsive UI in Flutter Web

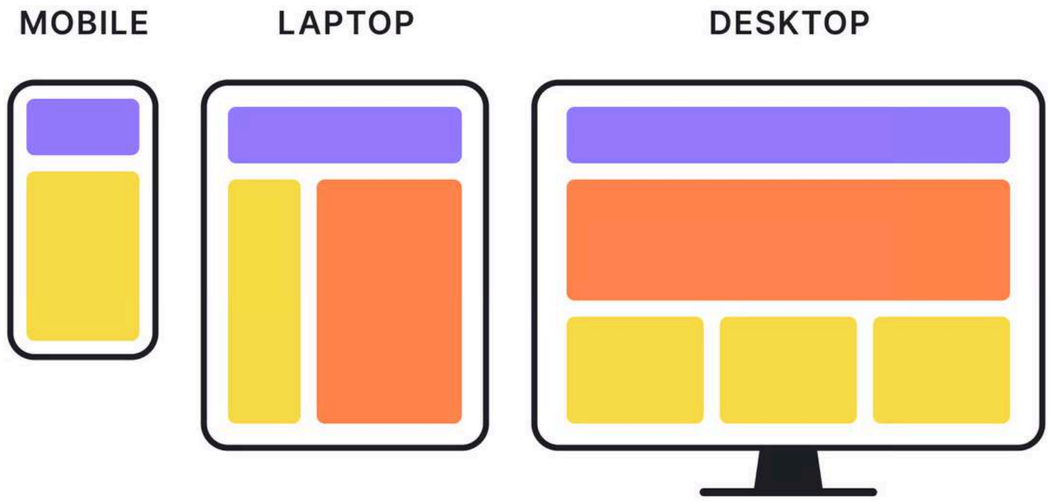
- Mobile layouts don’t scale automatically
- Screen width affects readability
- Layout structure must change by size
- Breakpoints are essential

adc

Responsive UI in Flutter Web

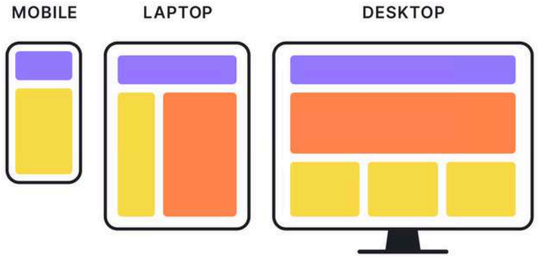
- LayoutBuilder
- MediaQuery breakpoints
- Grid & max-width containers

Adaptive UI In Flutter Web



Adaptive UI In Flutter Web

- Desktop and mobile users behave differently
- Input methods change (mouse vs touch)
- Navigation patterns differ
- Web has stronger UX expectations



adc

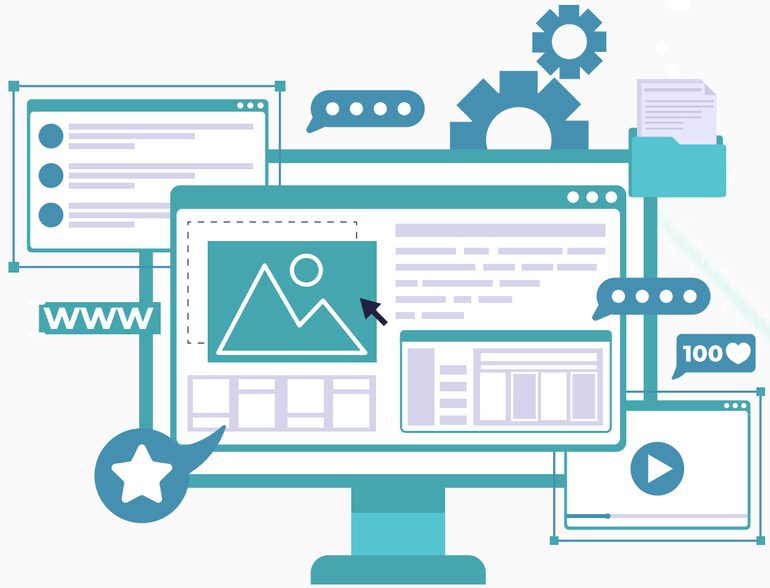
Routing & Navigation on Web

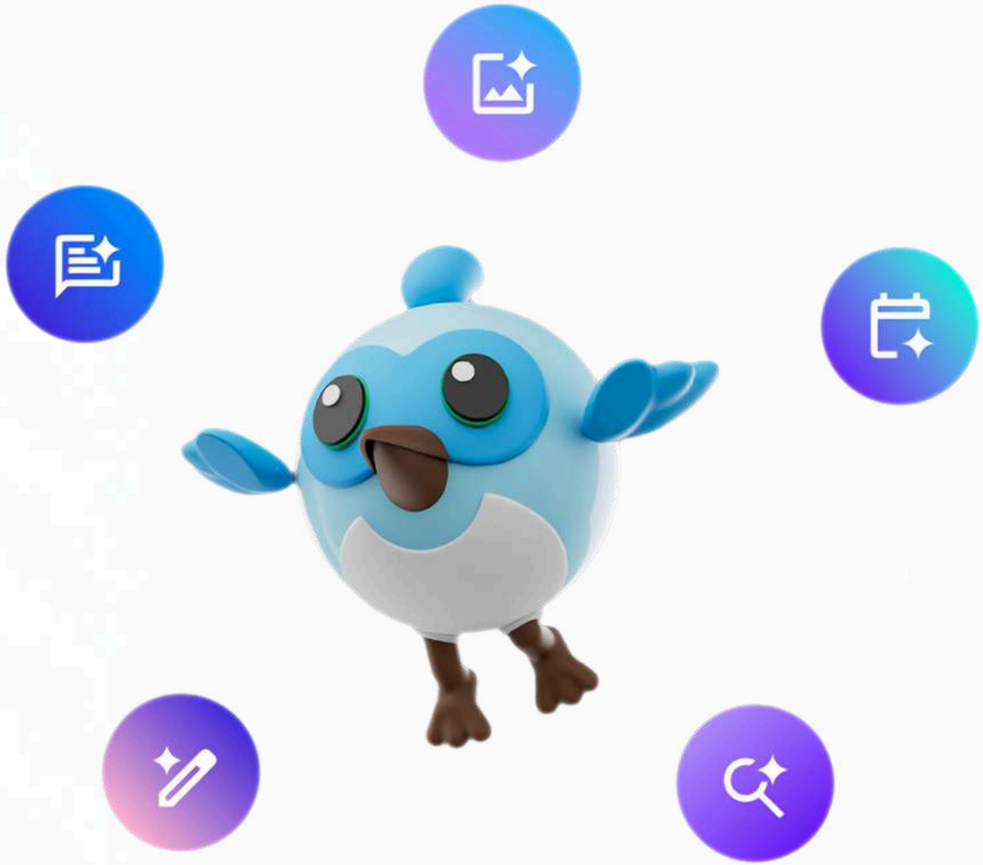
- URLs represent app state
- Browser refresh should not break navigation
- Deep linking is required
- Navigation must be predictable

adc

Routing & Navigation on Web

- go_router
- Named routes
- URL-based navigation



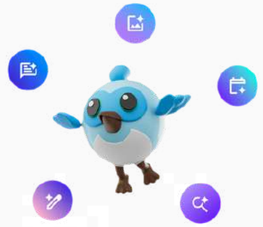


Flutter With Ai

Flutter With Ai

Cursor: Using AI Correctly in Flutter Projects

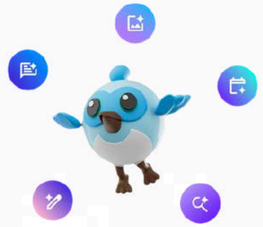
- Rules guide AI behavior
- Prompt type affects output quality
- Different models serve different tasks
- AI works best with structure



Flutter With Ai

Cursor: Using AI Correctly in Flutter Projects

- Rules guide AI behavior
- Prompt type affects output quality
- Different models serve different tasks
- AI works best with structure

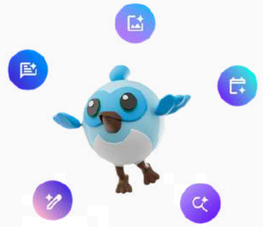


Flutter With Ai

Things to Know About Cursor

🧩 Rules

- Define coding style and architecture
- Enforce folder structure & patterns
- Prevent inconsistent code generation

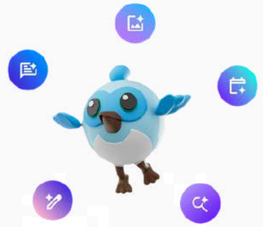


Flutter With Ai

Things to Know About Cursor

🧠 Prompt Types

- Ask → Quick questions & explanations
- “Why does this widget rebuild?”
- Plan → Architecture & feature design
- “Plan authentication flow for Flutter Web.”
- Agent → Multi-file changes
- “Add feature with repository, cubit, and UI.”
- Debug → Error tracing & fixes
- “Find why this API call fails on web.”



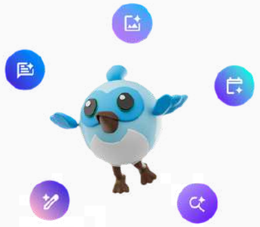
Flutter With Ai

Things to Know About Cursor

🤖 Model Selection

- Fast models → Small tasks, quick edits
- Strong reasoning models → Architecture & refactoring
- Large context models → Big codebases

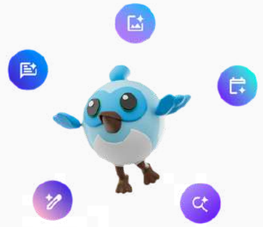
“Choosing the wrong model is like using a hammer for every problem.”



Flutter With Ai

Supabase MCP: AI That Understands Your Backend

- Reads tables, columns, and relationships
- Generates correct Supabase queries
- Understands Row Level Security (RLS)
- Prevents API hallucination

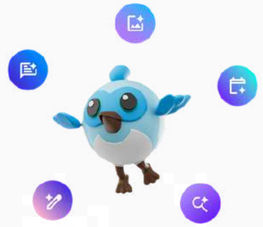


Flutter With Ai

Supabase MCP: AI That Understands Your Backend

Real Examples

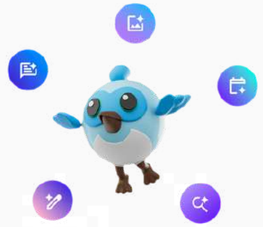
- “Generate Flutter models from Supabase tables.”
- “Write a query to fetch user-specific orders.”
- “Fix RLS permission issues.”
- “Create repository logic using Supabase client.”
- Result: Faster backend integration, fewer runtime errors.



Flutter With Ai

Figma MCP: From Design to Flutter UI

- Reads real Figma files
- Matches spacing, colors, and typography
- Converts components into Flutter widgets
- Reduces UI handoff friction

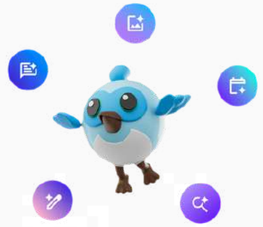


Flutter With Ai

Figma MCP: From Design to Flutter UI

Real Examples

- “Generate a Flutter screen from this Figma frame.”
- “Convert design components into reusable widgets.”
- “Match text styles and theme exactly.”
- “Extract design tokens into Flutter ThemeData.”
- Result: Faster UI implementation with fewer design mismatches.



adc

NETWORKING BLOCK

DISCUSSION CIRCLES

1. كوّنوا مجموعات من 4 إلى 6 أشخاص بالقرب منكم (بلا إختلاط)، حاول أن تنضم لمجموعة لا تحوي على معارفك.
2. ستظهر سؤال / فكرة للنقاش كل 5–6 دقائق على الشاشة.
3. احرصوا أن يتحدث كل شخص مرة واحدة على الأقل في كل جولة.
4. اجعلوا المشاركة مختصرة (30–60 ثانية) لكل شخص.
5. الرجاء احترام الآخرين وعدم المقاطعة.
6. عند تغيير السؤال، انتقلوا مباشرة إلى السؤال الجديد حتى لو لم ينتهي الموضوع السابق.

adc

1. What are you building now?

2. What architectural pattern (TDD, MVC, Bloc Pattern, ...) do you prefer, and why?

adc

3. How do you manage responsive layouts across devices?

4. A package that saved you time?

5. What do you want to learn next?

adc

OPEN NETWORKING

adc

FLUTTER FLASH

Flutter Platform Channels

Why and how we bridge Dart with native Android/iOS

Amjad Daher

www.linkedin.com/in/amjad-daher

Why use Platform Channels?

- Access native APIs (Battery, Sensors, Bluetooth, NFC)
- Use native SDKs
- Call system-level functionality

Types of Platform Channels

1. MethodChannel – Call native methods and receive results.
2. EventChannel – Receive continuous streams of data
3. BasicMessageChannel – Simple message passing

Example: Get Battery Level

Example: Get Battery Level

```
Flutter (Dart)
import 'package:flutter/services.dart';
class BatteryService {
 static const MethodChannel _channel =
 MethodChannel('com.example/battery');
 static Future<int> getBatteryLevel() async {
 final int batteryLevel =
 await
_channel.invokeMethod('getBatteryLevel');
 }
}
```

```
Android (Kotlin)
class MainActivity : FlutterActivity() {
 private val CHANNEL = "com.example/battery"
 override fun configureFlutterEngine(flutterEngine: FlutterEngine)
{MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL)
 .setMethodCallHandler { call, result →
 if (call.method == "getBatteryLevel") {
 result.success(85)
 } else {
 result.notImplemented()
 }
 }
 }
}
```

```
iOS (Swift)
let channel = FlutterMethodChannel(
 name: "com.example/battery",
 binaryMessenger: controller.binaryMessenger
)
channel.setMethodCallHandler { call, result
if call.method == "getBatteryLevel" {
 result(85)
 } else {
 result(FlutterMethodNotImplemented)
 }
}
```

Best Practices

- Use a unique channel name
- Handle errors properly
- Avoid heavy work on the main thread
- Prefer plugins for reusable logic

Thank you!

- Unlock Features: Access native power.
- Boost Performance: Delegate heavy tasks.
- Clean Architecture: Separate concerns.

Questions?

Connect with me on LinkedIn:
www.linkedin.com/in/amjad-daher

adc

FLUTTER TESTING

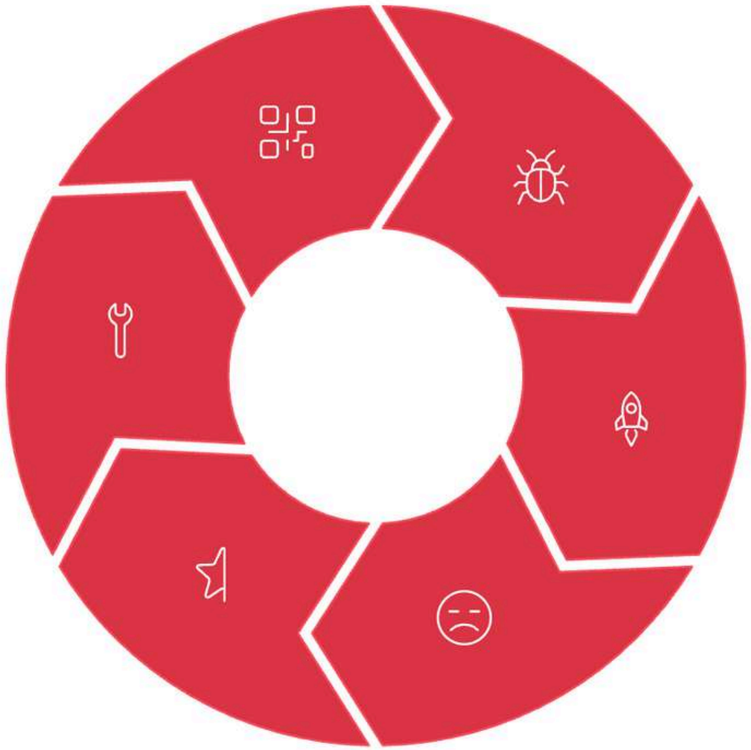
DOES YOUR APP HAVE BUG

Do you suffer from undiscovered bugs, after making changes to your app, due to lack of suffecient testing

Ahmed Rajab

Flutter Developer

The Problem: Without Tests



The Solution: With Flutter Tests

Fix Before Deploy

Clean Production

Happy Users + ⭐ High Ratings

Types of Automated Tests

Unit test	Widget Test	Integration Test
Scope: Individual functions, methods, business logic	Scope: Widget rendering and interactions	Scope: Complete user journeys
Key benefit: No UI dependencies	Key benefit: Test UI without full app	Key benefit: Real device/simulator testing
Focus: Pure logic testing	Focus: UI behavior validation	Focus: End-to-end validation
Speed: Lightning-fast	Speed: Fast	Speed: Slower but comprehensive

MOCKING DEPENDENCIES

- **Why Mock?**
- Isolate your tests from external dependencies that are slow, unreliable, or unavailable.

- **Common Mock Targets**
- REST API endpoints
- Database connections
- Authentication services
- Third-party SDKs
- File system operations

CODEMAGIC CI/CD INTEGRATION

Zero Configuration

Flutter projects work out of the box with intelligent defaults

Multi-Platform Builds

iOS, Android, web, and desktop from one pipeline

Automated Testing

Run unit, widget, and integration tests on every commit

THANK YOU

Connect with me on LinkedIn:
https://www.linkedin.com/in/ahmed-rajab-109272133/

FLUTTER RENDERING

Ever wondered why your Flutter app feels laggy?

FLUTTER DOESN'T RENDER WIDGETS DIRECTLY!

-Widget Tree: Immutable, describes WHAT

-Element Tree: Stable, connects Widget to RenderObject

-RenderObject Tree: Actual painting on screen

THE MAGIC PIPELINE!

- Widget declares the UI (immutable)

- Element manages lifecycle & state

- RenderObject: Layout, Paint, Composite

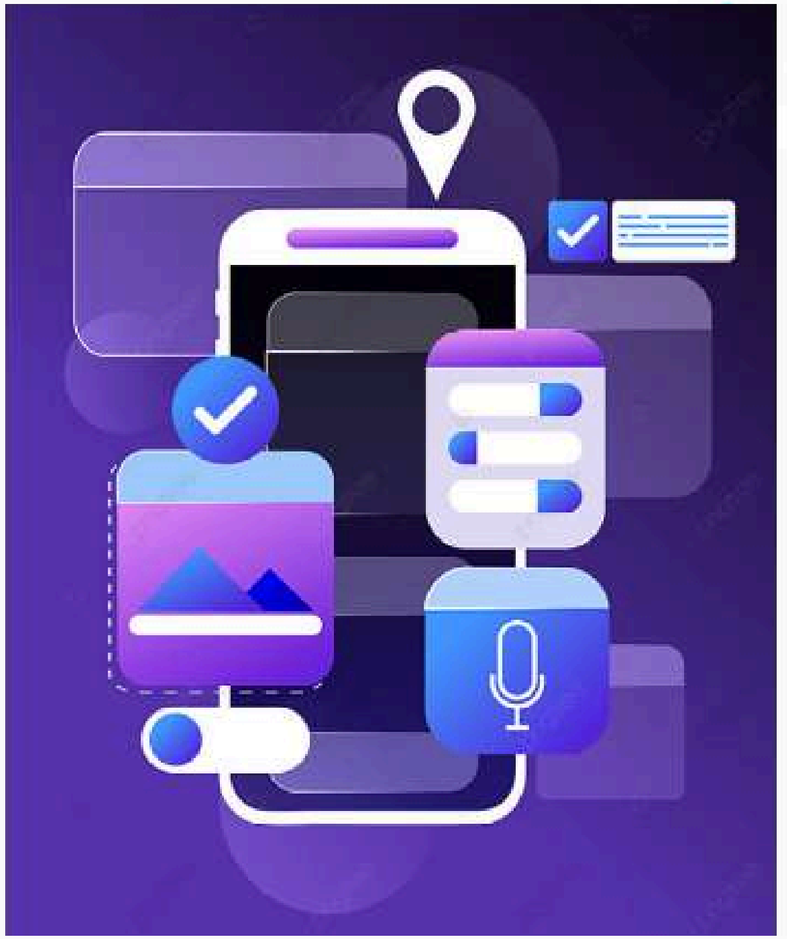
adc

RENDERING IN 4 PHASES

- LAYOUT: Calculate size & position

- PAINT: Draw on canvas

- COMPOSITE: Combine layers



THE SECRET WEAPON: RepaintBoundary

- Without: Every scroll = Repaint EVERYTHING!

- With: Each item isolated & independent

- Result: Smooth 60 FPS!

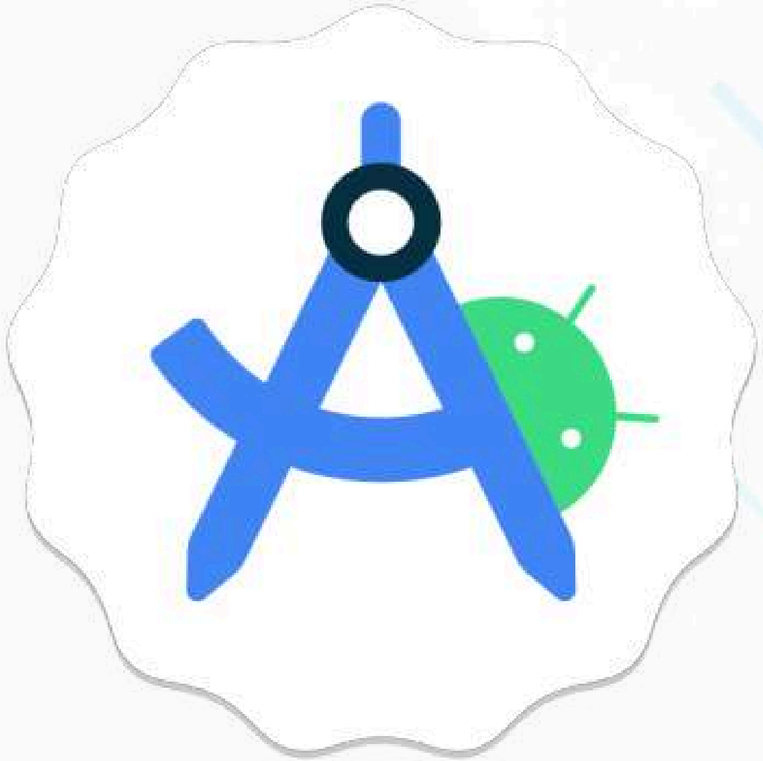
adc

Code example

- ListView.builder(

- itemBuilder: (context, index) {

- return RepaintBoundary(child: Item());}



3 GOLDEN RULES

- Use const constructors everywhere
- Isolate changing parts with RepaintBoundary
- Measure with DevTools - Don't guess!

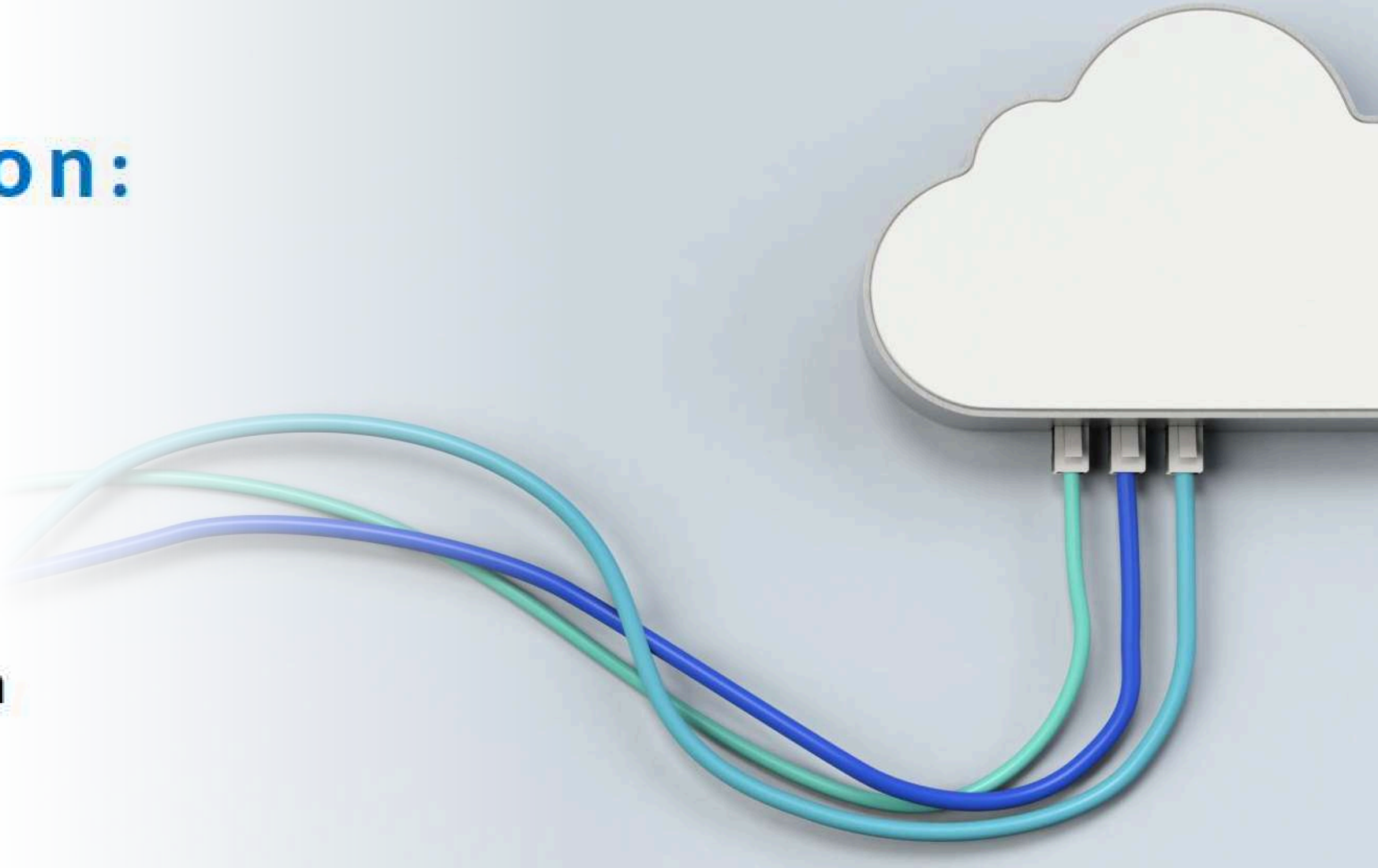
Thank you

Mahmood Rihanya
Flutter Developer & Package Publisher
https://www.linkedin.com/in/mahmoodflutter

FLUTTER HARDWARE CONNECTION

Introduction:

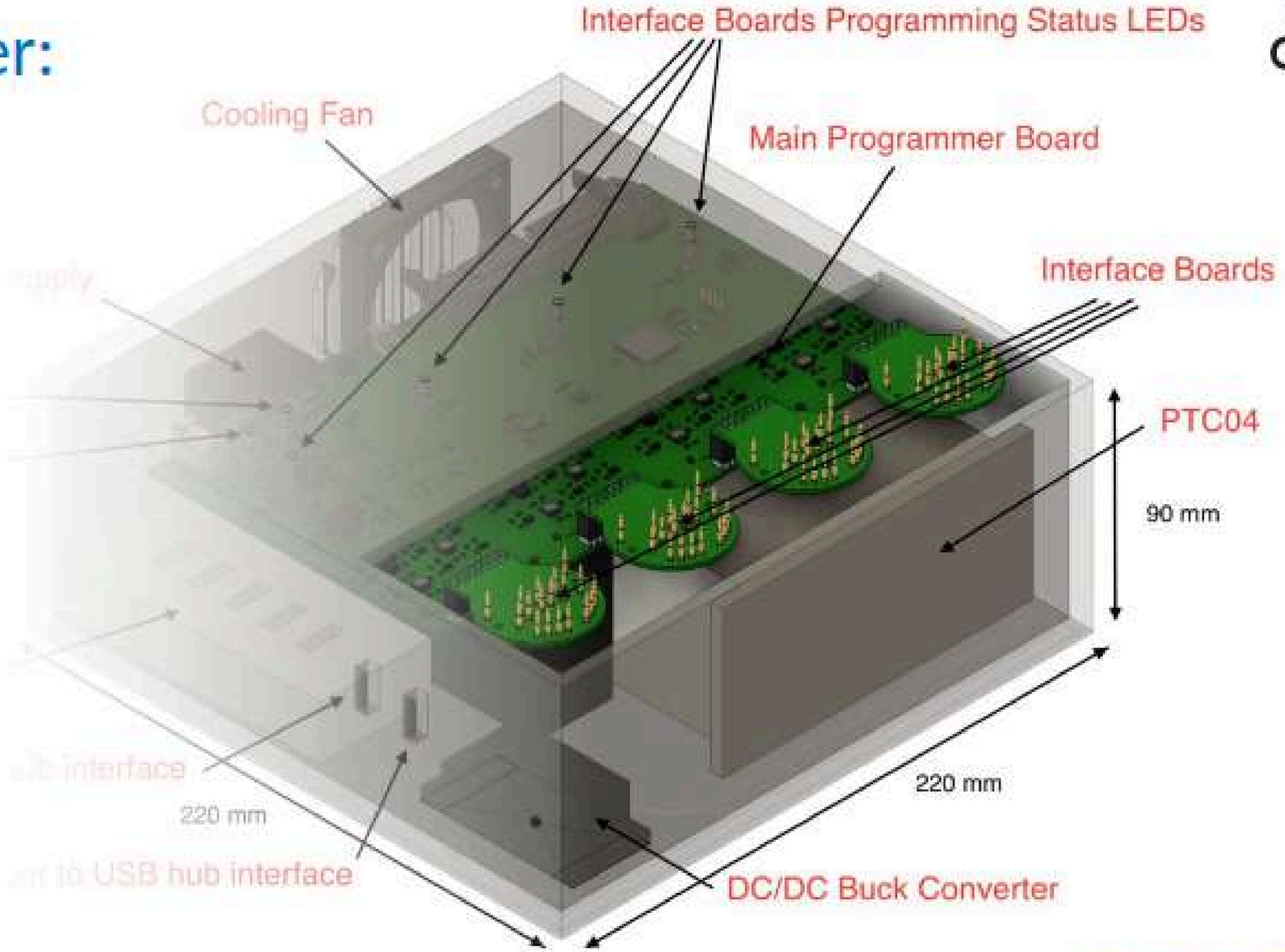
- TPMS Programmer
- TPMS Hardware
- USB Connection
- BLE Connection
- Desktop Application



1-TPMS Programmer:

- It is a specialized industrial programmer designed for programming a TPMS (Tire Pressure Monitoring System) module that measures tire pressure in trucks in real time and gives us the coordinates of the truck, in addition to different types of measurements that help the driver know the status of their truck.

- It was ordered after designing the TPMS on a large scale, so they need it for mass production.



TPMS Programmer Enclosure Concept v0.1

Front/Top/Side View

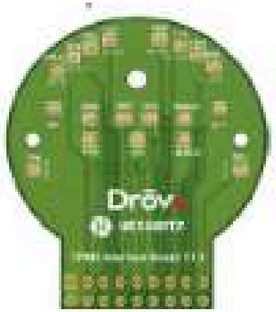
1-TPMS Programmer:

The main functionality for the TPMS programmer:

- Electrical Test
- MCU Programming
- MCU Lock
- Sensor Programming
- Sensor, which we had 2 sensors

2-TPMS Hardware

•The hardware contains the circuit responsible for providing ETR testing for the module and establishing connections to two types of universal programmers to assist in programming the MCU and the sensor.



adc

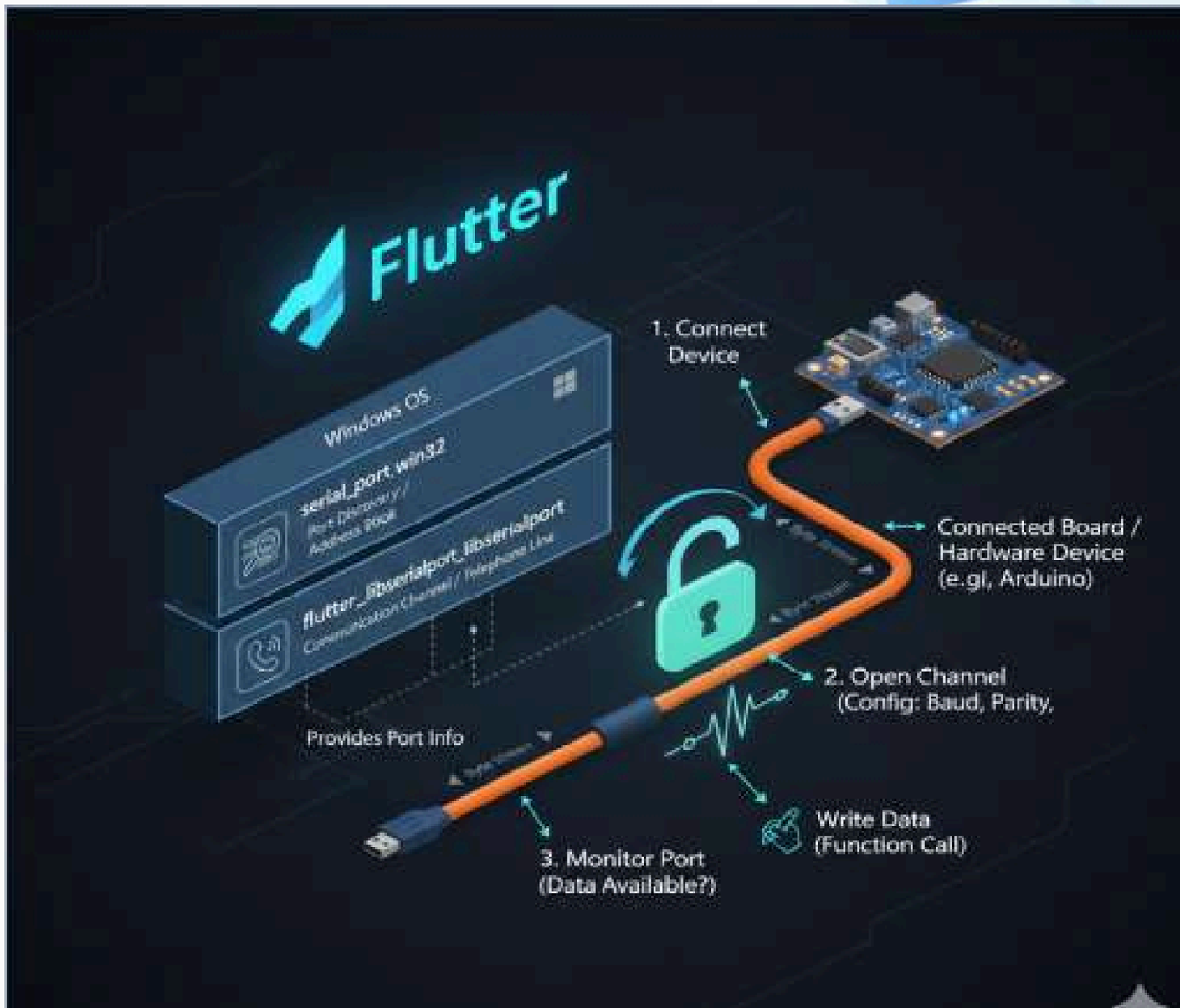
3-USB Connection

Libraries:

1. serial_port_win32: This library defines the available serial ports.
2. Flutter_libserialport: This library is used for reading and writing to the serial port.

How it works:

1. Connect the device you want to use.
2. Open a serial communication channel for reading and writing data.
3. Monitor the port to check if there is any data being transmitted. If you want to send data, use a function to write to the connected board.



4-BLE Connection

Libraries:

Flutter_blue_plus.

How it works:

1. Ensure that the app has permission for Bluetooth and location.
2. Search for BLE devices.
3. Connect to the desired device.
4. Transfer data between the two peers.

1

Mac: 60:A4:34:70:00:00

BarCode: 1111111111111

- All
- ETR
- MCU Prog
- MCU Lock Prog
- Sensor Prog
- MCU Unlock Prog

2

Mac: 60:A4:34:70:02:02

BarCode: 2222222222222

- All
- ETR
- MCU Prog
- MCU Lock Prog
- Sensor Prog
- MCU Unlock Prog

3

Mac: 60:A4:34:70:03:03

BarCode: 3333333333333

- All
- ETR
- MCU Prog
- MCU Lock Prog
- Sensor Prog
- MCU Unlock Prog

4

Mac: 60:A4:34:70:00:01

BarCode: 1234567891234

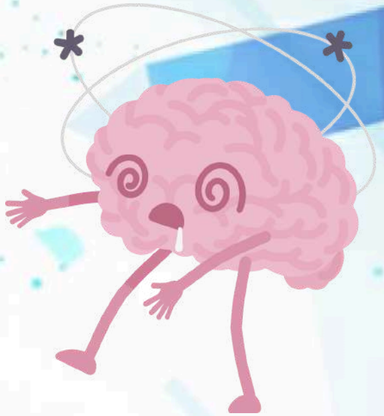
- All
- ETR
- MCU Prog
- MCU Lock Prog
- Sensor Prog
- MCU Unlock Prog

Result	ETR	MCU Prog	MCU Lock Prog	Sensor 1 Prog	Sensor 2 Prog
1	true	true	true	true	true
2	false	true	-	true	true
3	-	true	-	-	-
4	-	true	true	-	-

Thanks for listening

CODE GEN MAGIC : SUPERCHARGE YOUR FLUTTER WORKFLOW

adc



The Boilerplate Fatigue

Every Flutter developer knows the pain. Let's face it—we didn't become engineers to type the same code over and over.

💀 Repetitive Drudgery

Writing hashCode, ==, and copyWith manually is exhausting and soul-crushing.

⚠️ Human Error

One tiny typo in a JSON key string can crash your entire app at runtime. Ouch.

🕸️ Maintenance Hell

Refactoring a single field breaks code in 5 different files. Your future self will hate you.

Abdulkareem Attar

Software Engineer | Flutter Developer

adc

The Engine: build_runner

01

The Concept

It watches your code and transforms **Annotations** into **Implementation** automatically.

02

The Command

```
flutter pub run build_runner watch -d
```

or

```
flutter pub run build_runner build -d
```

03

The Output

Generates the "dirty work" files (.g.dart), keeping your main code pristine and readable.

JSON_SERIALIZABLE

Data Modelling Made Easy

before_json_serializable ✗

```dart
class User {
  final String name;
  final int age;

  User({required this.name, required this.age});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      name: json['name'] as String,
      age: json['age'] as int,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'name': name,
      'age': age,
    };
  }
}
```

after_json_serializable ✓

```dart
import 'package:json_annotation/json_annotation.dart';
part 'user.g.dart';

@JsonSerializable()
class User {
  final String name;
  final int age;

  User({required this.name, required this.age});

  factory User.fromJson(Map<String, dynamic> json) =>
   _$UserFromJson(json);
  Map<String, dynamic> toJson() => _$UserToJson(this);
}
```

Freezed

Data Classes & Immutability

before_freezed ✗

```dart
class Product {
  final String id;
  final String name;

  Product({required this.id, required this.name});

  Product copyWith({String? id, String? name}) {
    return Product(
      id: id ?? this.id,
      name: name ?? this.name,
    );
  }

  @override
  bool operator ==(Object other) {
    if (identical(this, other)) return true;
    return other is Product &&
    other.id == id && other.name == name;
  }

  @override
  int get hashCode => id.hashCode ^ name.hashCode;
}
```

after_freezed ✓

```dart
import 'package:freezed_annotation/freezed_annotation.dart';
part 'product.freezed.dart';

@freezed
class Product with _$Product {
  const factory Product({
    required String id,
    required String name,
  }) = _Product;
}
```

Retrofit

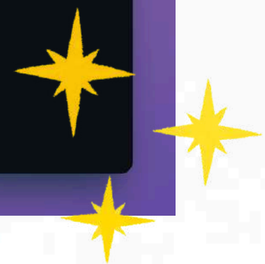
Type-safe Rest-API Calls

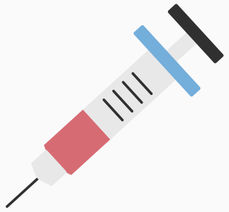
before_retrofit ✗

```dart
import 'package:dio/dio.dart';

class ApiService {
  final Dio _dio;

  ApiService(this._dio);

  // 1. جلب بيانات (Get) - التحويل يدوي ومسار نصي
  Future<User> getUser(String id) async {
    final response = await _dio.get('/users/$id');
    return User.fromJson(response.data);
  }

  // 2. إرسال بيانات (Post) - يجب تحويل الكائن لـ Map يدوياً
  Future<User> createUser(User user) async {
    final response = await _dio.post(
      '/users',
      data: user.toJson(),
    );
    return User.fromJson(response.data);
  }

  // 3. تحديث بيانات (Put)
  Future<User> updateUser(String id, User user) async {
    final response = await _dio.put(
      '/users/$id',
      data: user.toJson(),
    );
    return User.fromJson(response.data);
  }

  // 4. حذف بيانات (Delete)
  Future<void> deleteUser(String id) async {
    await _dio.delete('/users/$id');
  }

  // 5. رفع ملفات (Multi-part) - كود معقد جداً يدوياً
  Future<String> uploadImage(File file) async {
    String fileName = file.path.split('/').last;
    FormData formData = FormData.fromMap({
      "file": await MultipartFile.fromFile(file.path, filename: fileName),
    });

    final response = await _dio.post("/upload", data: formData);
    return response.data['url'];
  }
}
```

after_retrofit ✓

```dart
import 'package:dio/dio.dart';
import 'package:retrofit/retrofit.dart';

part 'api_service.g.dart';

@RestApi(baseUrl: "https://api.example.com")
abstract class ApiService {
  factory ApiService(Dio dio, {String baseUrl}) = _ApiService;

  // 1. جلب بيانات (Get)
  @GET("/users/{id}")
  Future<User> getUser(@Path("id") String id);

  // 2. إرسال بيانات جديدة (Post)
  @POST("/users")
  Future<User> createUser(@Body() User user);

  // 3. تحديث بيانات بالكامل (Put)
  @PUT("/users/{id}")
  Future<User> updateUser(@Path() String id, @Body() User user);

  // 4. تحديث جزئي للبيانات (Patch)
  @PATCH("/users/{id}")
  Future<User> updateNickname(@Path() String id,
  @Field("nickname") String name);

  // 5. حذف بيانات (Delete)
  @DELETE("/users/{id}")
  Future<void> deleteUser(@Path() String id);

  // 6. إرسال ملفات / صور (Multi-part)
  @POST("/upload")
  @MultiPart()
  Future<String> uploadImage(@Part() File file);
}
```





Injectable

Dependency Injection

before_injectable ❌

```dart
final getIt = GetIt.instance;

void setup() {
  // Manual registration (Order matters!)
  getIt.registerFactory<AuthService>(() => AuthService());
  getIt.registerLazySingleton<Database>(() => Database());
  //more!
}
```

after_injectable ✅

```dart
import 'package:injectable/injectable.dart';

// 1. New instance every time it's requested
@injectable
class AuthService {}

// 2. Single instance, created only when needed
@lazySingleton
class DatabaseService {}

// 3. Single instance, created immediately on app start
@singleton
class AppConfig {}

// Setup generated by injectable
@InjectableInit()
void configureDependencies() => getIt.init();
```

COMING SOON

The Future: Dart Macros

Static Metaprogramming

The Dart team is building something revolutionary:

- **No build_runner** — macros run at compile time
- **No .g.dart files** — code exists only in memory
- **Real-time generation** — instant feedback in your IDE

Code generation, directly in the compiler.

Abdulkareem Attar

Software Engineer | Flutter Developer

Connect with me on LinkedIn:

https://www.linkedin.com/in/abdulkareemattar/

Let’s Build the Future of Aleppo’s Tech Scene Together!

STAY CONNECTED!

aleppo.dev